

CURSO TÉCNICO EM INFORMÁTICA

Linguagem de Programação Estruturada

Aula 4 — Programação Estruturada, Paradigmas e Mercado

Professor: Rafael de Olanda · Turma: TI1F



Agenda da aula 4

- Recap rápido da aula 3
- 1. Programação estruturada: decomposição, funções e escopo
- 2. Estruturada vs Orientada a Objetos (conceitos, sem aprofundar)
- 3. Termos de mercado: frontend, backend, API, CRM, ERP, PDV
- 4. IA e o mercado de programação hoje
- Revisão geral e Avaliação Teórica 1

Recap — o que vimos na aula 3

- if/else, else if e switch: decidir entre dois ou mais caminhos
- for, while e do-while: repetir um bloco, cada um no seu momento certo
- Fluxograma → pseudocódigo → código: o mesmo raciocínio, três representações
- Hoje: como organizamos programas maiores, e onde tudo isso se encaixa no mercado



1

Programação Estruturada

Organizando programas maiores: decomposição, funções e escopo

Decomposição top-down

Dividir um problema grande em pedaços menores e mais fáceis de resolver

Problema: "Fazer o sistema de vendas de uma loja"

**Cadastrar
produtos**

**Registrar
vendas**

**Calcular
totais**

**Emitir
relatórios**

Cada uma dessas partes menores ainda pode ser dividida de novo — "Registrar vendas", por exemplo, quebra em "ler produto", "ler quantidade", "calcular subtotal". É a mesma ideia da decomposição que vocês já viram em Lógica, agora aplicada a um programa inteiro, não só a um exercício.

Modularização — dividindo em funções

Cada "pedaço" da decomposição vira uma sub-rotina que pode ser chamada quando precisar

Pseudocódigo

```
INÍCIO
  SUBROTINA calcularMedia(nota1, nota2)
    RETORNE (nota1 + nota2) / 2
  FIM_SUBROTINA

  LER n1, n2
  media <- calcularMedia(n1, n2)
  ESCREVER media
FIM
```

...e como isso aparece em C e em Pascal

Só uma prévia — o assunto "função" será aprofundado de verdade na aula 9

C

```
float calcularMedia(float n1, float n2) {  
    return (n1 + n2) / 2;  
}  
  
int main() {  
    float n1, n2, media;  
    scanf("%f %f", &n1, &n2);  
    media = calcularMedia(n1, n2);  
    printf("%.2f", media);  
}
```

Pascal

```
function CalcularMedia(n1, n2: real): real;  
begin  
    CalcularMedia := (n1 + n2) / 2;  
end;  
  
var n1, n2, media: real;  
begin  
    readln(n1, n2);  
    media := CalcularMedia(n1, n2);  
    writeln(media:0:2);  
end.
```

Escopo de variável — local vs global

Onde uma variável existe e pode ser usada dentro do programa

Pseudocódigo

```
INÍCIO
    total <- 0                // variável GLOBAL

    SUBROTINA somar(valor)
        local <- valor * 2    // variável LOCAL
        total <- total + local // só a global sobrevive fora daqui
    FIM_SUBROTINA
FIM
```

Escopo — em C e em Pascal

A mesma ideia de agora, escrita nas duas linguagens (só pra reconhecer — não precisa decorar ainda)

C

```
int total = 0;           // global

void somar(int valor) {
    int local = valor * 2; // local
    total = total + local;
}

int main() {
    somar(5);
    printf("%d", total);
}
```

Pascal

```
var total: integer;      { global }

procedure Somar(valor: integer);
var local: integer;      { local }
begin
    local := valor * 2;
    total := total + local;
end;

begin
    total := 0;
    Somar(5);
    writeln(total);
end.
```



2

Estruturada vs Orientada a Objetos

Dois jeitos de organizar a lógica de um programa

O que é Orientação a Objetos?

Um paradigma diferente — vocês vão aprofundar isso numa disciplina própria mais à frente

Classe

Um "molde" ou planta baixa: descreve que características (atributos) e comportamentos (métodos) algo vai ter.

Exemplo: a classe Carro define que todo carro tem cor, modelo e velocidade, e sabe acelerar e frear.

Objeto

Uma instância real, construída a partir da classe. Se Carro é a planta baixa, "meu Fusca azul" é um objeto concreto — com cor, modelo e velocidade específicos, criado a partir daquele molde.

Na programação estruturada, dados e funções ficam separados. Na orientação a objetos, cada objeto já carrega junto os seus próprios dados e as ações que sabe fazer com eles.

Os pilares da OO — nível de prévia

Três ideias que vocês vão aprofundar de verdade na disciplina de Orientação a Objetos

Herança

Uma classe pode "herdar" características de outra. A classe `VeiculoEletrico` pode herdar tudo de `Carro` e só acrescentar o que for diferente (bateria, autonomia), sem reescrever o resto.

Polimorfismo

Objetos diferentes podem responder de formas diferentes ao mesmo comando. Pedir para "emitir som" a um `Cachorro` e a um `Gato` produz resultados diferentes, mesmo sendo a mesma instrução.

Encapsulamento

O objeto protege seus próprios dados, expondo só o necessário pro resto do programa. Ninguém mexe diretamente no "motor" de um `Carro` — usa o método `acelerar()`, que sabe fazer isso com segurança.

O mesmo problema, dois estilos

Calcular a área de figuras geométricas — de forma conceitual, sem código completo

Estruturada

Uma função `calcularArea(tipo, dados)` recebe QUAL figura é e decide o cálculo certo por dentro, geralmente com um SE ou ESCOLHA:

```
SE tipo = "quadrado" ENTÃO
  area <- lado * lado
SENÃO SE tipo = "círculo" ENTÃO
  area <- 3.14 * raio2
```

Orientada a Objetos

Cada figura é a sua própria classe, e cada uma sabe calcular a própria área — não existe um SE central decidindo por todas:

```
classe Quadrado:
  area() -> lado * lado

classe Circulo:
  area() -> 3.14 * raio2
```



3

Termos de Mercado

Vocabulário que aparece em toda vaga e reunião de TI

Frontend vs Backend

As duas metades de praticamente todo sistema

Frontend

A parte que o usuário vê e interage: telas, botões, formulários. Roda no navegador ou no aplicativo do próprio usuário. Exemplos de tecnologia: HTML, CSS, JavaScript — que vocês já viram na outra disciplina.

Backend

A parte que roda no servidor, longe dos olhos do usuário: regras de negócio, banco de dados, autenticação. É aqui que linguagens estruturadas como C, e outras como Java, Python e PHP, historicamente reinam.

Analogia: num restaurante, o salão (onde você faz o pedido e recebe o prato) é o frontend; a cozinha (onde o prato é de fato preparado) é o backend. O cliente nunca vê a cozinha, mas é ela que faz tudo funcionar.



API

O "garçom" que leva pedidos entre dois sistemas diferentes

API (Application Programming Interface) é um conjunto combinado de "pedidos" que um sistema aceita fazer para outro — sem que um precise saber como o outro funciona por dentro.

Exemplo do dia a dia: um app de previsão do tempo não sabe medir a temperatura sozinho — ele pede essa informação pra API de um serviço meteorológico, recebe a resposta pronta e só mostra na tela.

Gancho com frontend/backend: é muito comum o frontend "conversar" com o backend justamente através de uma API — o garçom levando o pedido do salão pra cozinha e trazendo o prato de volta.

CRM, ERP e PDV

Sistemas que praticamente toda empresa usa — e que sustentaram décadas de programação estruturada

CRM

Customer Relationship Management. Organiza o relacionamento com clientes: contatos, histórico de vendas, propostas em andamento.

ERP

Enterprise Resource Planning. Integra os processos internos da empresa: estoque, financeiro, folha de pagamento, compras — tudo num só sistema.

PDV

Ponto de Venda. O sistema do caixa: registra a venda, calcula o total, emite o cupom fiscal — o que vocês veem toda vez que compram algo numa loja.

Gancho histórico: por décadas, sistemas de CRM, ERP e PDV foram construídos majoritariamente em linguagens estruturadas — C e COBOL à frente. Muito sistema desses continua rodando em produção até hoje, o que faz programação estruturada ser uma habilidade valorizada bem além da sala de aula.



4

IA no Mercado de Programação

O que muda, e o que continua dependendo de fundamento

IA no dia a dia de quem programa

- Copilot, ChatGPT e assistentes parecidos: sugerem código, completam trechos, explicam erros e ajudam a escrever mais rápido
- Já fazem parte do fluxo de trabalho de boa parte dos profissionais de TI — não é mais "o futuro", é o presente
- Mas: a IA sugere código baseado em padrões — quem decide se aquele código está certo, resolve o problema certo, e é seguro, ainda é uma pessoa

O que muda, e o que continua dependendo de fundamento

A IA acelera quem já entende lógica — não substitui entender lógica

O que a IA acelera

Escrever código repetitivo, lembrar sintaxe exata, gerar um primeiro rascunho de função, explicar uma mensagem de erro, sugerir nomes e pequenas melhorias.

O que continua exigindo você

Entender o problema de verdade, decidir a lógica certa, revisar se o código faz sentido, testar, e responder 'por que esse código está errado?' — tudo isso é exatamente o que vocês estão treinando nesta disciplina.

Mensagem central: quem entende lógica, sintaxe e semântica consegue USAR a IA como ferramenta e revisar o que ela sugere. Quem não entende, só copia sem saber se está certo.



Revisão geral — aulas 1 a 4

Antes da prova: um resumo rápido de cada aula

Aula 1	Algoritmo, abstração, alto/baixo nível, compilado/interpretado, C e Pascal
Aula 2	Sintaxe vs semântica, tipos de dados, variáveis, constantes, arrays (conceito)
Aula 3	if/else, switch, for/while/do-while, fluxograma → pseudocódigo → código
Aula 4	Decomposição, funções, escopo, estruturada vs OO, termos de mercado, IA

Avaliação Teórica 1

Cobrando o conteúdo das aulas 1 a 4

O que cai:

história e fundamentos, sintaxe/semântica, tipos de dados, estruturas de condição e repetição, programação estruturada e os conceitos de mercado vistos hoje.

Formato:

questões objetivas e de análise de código/pseudocódigo — sem necessidade de compilador, como foi até agora.



Aula 4 — Fim do bloco teórico!

Aula 5: instalamos a IDE e escrevemos nosso primeiro programa em C.